

Exercice 1. – Compter dans un arbre –

On considère un arbre *orienté* (un site distingue son père de ses fils) dans un système asynchrone. On souhaite calculer des informations relatives à cet arbre. Dans chacune des questions suivantes, on vous donne l'information que l'on souhaite calculer et vous devez écrire un algorithme distribué tel que, à la fin de l'exécution, la *racine* de l'arbre connaît l'information désirée. On suppose que les initiateurs sont les feuilles de l'arbre.

1. Calculer le nombre de *feuilles* (*i.e.*, le nombre de nœuds sans fils) contenues dans l'arbre dans le cas où il est possible de mettre des valeurs dans les messages. Indiquer le mode de terminaison (implicite ou explicite, auquel cas vous indiquerez la (ou les) action(s) STOP_GLOBAL/STOP_LOCAL). Donner la complexité en messages et en temps de votre algorithme, dans le pire cas.
2. Calculer le nombre de *feuilles* contenues dans l'arbre dans le cas où il est interdit de mettre des valeurs dans les messages.
 - (i) Proposer dans un premier temps, un algorithme avec terminaison implicite. Donner un exemple d'arbre dans lequel la complexité en message est $n - 1$, puis un autre exemple dans lequel la complexité est $(n/2)^2$ (avec n , le nombre de nœuds dans l'arbre). En déduire la complexité en messages de votre algorithme, dans le pire cas. Quelle est sa complexité en temps dans le pire cas ?
 - (ii) Proposer dans un second temps, un algorithme avec terminaison explicite, en supposant des canaux FIFO. Puis, donner un exemple illustrant la nécessité d'avoir des canaux FIFO. Quelle est la complexité en messages et en temps de cet algorithme, dans le pire cas.
3. Calculer le nombre de nœuds *spéciaux* contenus dans l'arbre. Un nœud spécial est un nœud interne avec *exactement* un de ses fils qui est une feuille.

Connaissance :

père_i : implicitement : } racine : $\text{père}_i = \emptyset$
 fils_i : } feuille : $\text{fils}_i = \emptyset$

2. 1) 隐式终止. 不停向上传. 只有根结点处统计数据
 消息中不能加数字

Connaissance :
 père_i
 fils_i

Variable: nbfeuille.
 Message: Msg

Code: Initialisé sur la feuille.
 Si $\text{fils}_i = \emptyset$ alors
 Envoyer Msg à père_i
 stop local

Sur la réception de Msg de j
 Si $\text{père}_i \neq \text{null}$
 Envoyer Msg à père_i
 Sinon
 nbfeuille ++

消息复杂度 $n-1$ $(n/2)^2$ 最坏(星)时间复杂度

TD4
 ex 1.1. 计算叶子的数量 叶子→根
 消息中可以加数字 Msgc=1

Connaissance:
 père_i
 fils_i

Variable:
 Nbregu- i : nb. de message reçu initialisé à 0.
 Nbfeuille: compteur. initialisé à 0

Message: Msgcv nb de feuille du sous-arbre de i

Code: (initialisé sur la feuille)
 Si $\text{fils}_i = \emptyset$ alors
 Envoyer Msgc=1 à père_i
 stop local

sur la réception de Msgcv de j
 Nbregu- i ++
 Nbfeuille ← Nbfeuille + v.
 Si $\text{Nbregu-}i = \text{fils}_i$ alors
 si $\text{père}_i = \text{null}$
 stop global
 Sinon
 Envoyer Msg (Nbfeuille) à père_i
 stop local

3. 显示 用 fin()

Connaissance: fil-i

pere-i

Variable: nb-feuille

nb-regu

Msg: est-feuille()

fin()

Initialisation sur les feuilles

Si fil-i == 0

Envoyer est-feuille() à pere-i

Envoyer fin() à pere-i

stop local

Sur la reception est-feuille() de j

Si pere-i == null

nb-feuille++

Sinon envoyer est-feuille() à pere-i

Sur la reception de fin() de j

nb-regu++

Si nb-regu == (fil-i) et pere-i != null

Envoyer fin() à pere-i

stop local

用 fin()

消息中不计数

3. 特殊结构: 子节点中只有一个叶子节点

Connaissance: fil-i

pere-i

Variable

nb-feuille

nbNs:

cpt

Msg

MycnbNs()

est-feuille()

Initialisation sur les feuilles

Envoyer est-feuille() à pere-i

stop local

Reception de est-feuille()

nb-feuille++

cpt++

Si cpt == fil-i

Si nb-feuille == 1

nbNs++

Si pere-i == null

stop global

Sinon Envoyer MycnbNs() à pere-i

stop local

sur la reception de Myc(k) de j

cpt++

nbNs = nbNs + k

Si cpt == fil-i

Si nb-feuille == 1

nbNs++

Si pere-i == null

stop global

Sinon envoyer MycnbNs() à pere-i stop local

叶子计数器++

计数器++

计数器 = 儿子数

叶子计数器 = 1

nbNs 1

在父节点(根)

Stop

在父节点

Msgc nbNs() 到父节点

计数++

nbNs += k

计数器 = 儿子数

...